PROJET MACHINE LEARNING

Classification de Malwares par Analyse Statique et Apprentissage Supervisé

Rapport de Synthèse Technique

Auteur :
Amadou Korka DIALLO /
Seydina Mouhamed Rassoul GOUDIABY

Date :
28 Mars 2026

Année Universitaire 2025-2026

Table des matières

1 Introduction et Contexte **2**

2 Exploitation du Dataset et Prétraitement **2**
2.1 Génération et Structure des Données 2
2.2 Prétraitement et Normalisation 4

3 Étude Comparative Multimodèle **4**
3.1 Les Modèles Évalués 4
3.2 Résultats de l'Évaluation 4

4 Optimisation du Modèle Champion **5**

5 Déploiement et Inférence **6**

6 Conclusion **6**

1 Introduction et Contexte

La prolifération des menaces informatiques sous Windows nécessite des outils de détection capables de traiter des fichiers au format Portable Executable (PE). L'approche traditionnelle par signatures montrant ses limites face aux malwares polymorphes ou inconnus (Zero-day), ce projet exploite le **Machine Learning** pour identifier des comportements malveillants de manière heuristique.

L'analyse employée est **statique** : les caractéristiques structurelles du binaire sont extraites sans l'exécuter, évitant tout risque d'infection de l'environnement d'analyse.

2 Exploitation du Dataset et Prétraitement

2.1 Génération et Structure des Données

Le jeu de données a été construit en simulant les caractéristiques métadonnées de fichiers PE sains (Label 0) et malveillants (Label 1). Pour chaque exécutable, 5 descripteurs (features) ont été retenus :

- **SizeOfImage** : La taille en mémoire de l'exécutable.
- **Entropy** : L'entropie de Shannon calculée sur les octets du fichier. Une entropie élevée (ex. > 7.0) est souvent corrélée à un code condensé (packé) ou chiffré, technique courante chez les malwares.
- **NumberOfSections** : Le nombre de sections définies dans l'en-tête PE.
- **Imports** : Le nombre de fonctions importées depuis des DLL externes. Un nombre anormalement faible ou ciblé peut être suspect.
- **Exports** : Le nombre de fonctions exportées (surtout pertinent pour les DLL).

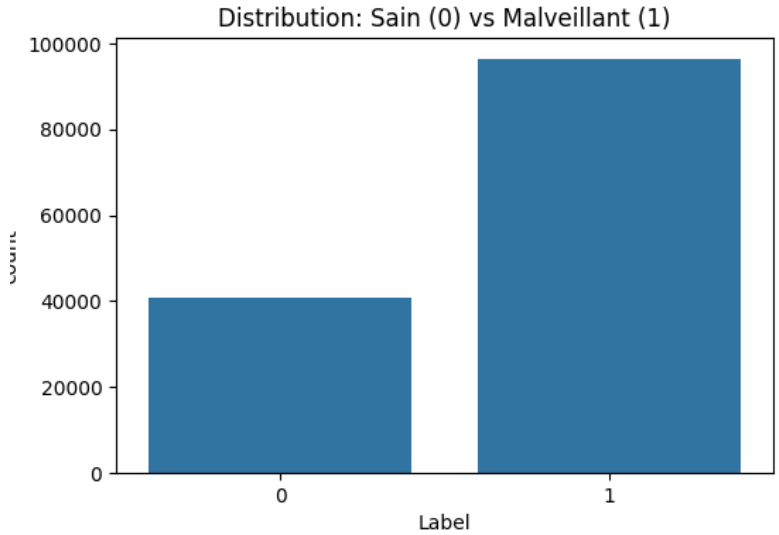

FIGURE 1 – *Distribution équilibrée des classes (Sain vs Malveillant)*

L'analyse exploratoire a mis en évidence des comportements distincts selon la nature du fichier. Par exemple, la distribution de l'entropie montre une nette tendance des malwares à présenter une entropie élevée (souvent supérieure à 7), indiquant possiblement

l'utilisation de techniques de packing (compression/chiffrement) destinées à obfusquer le code malveillant.

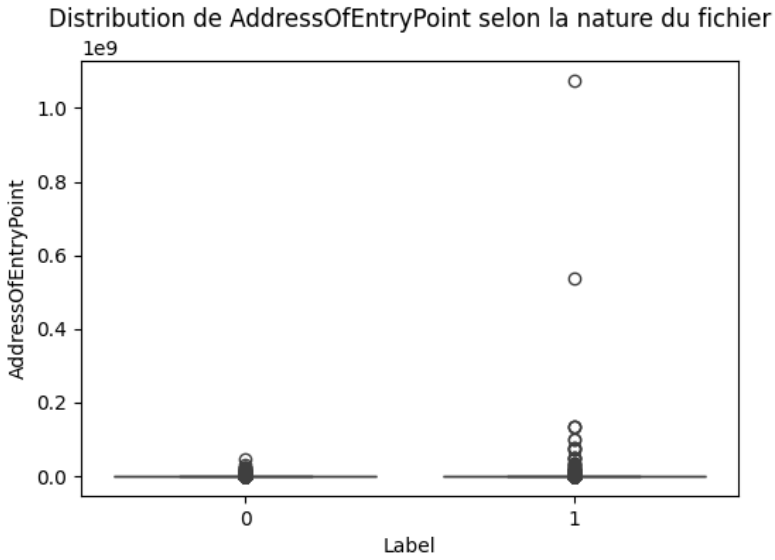

FIGURE 2 – *Distribution de l'entropie selon la nature du fichier*

Afin d'évaluer la pertinence de chaque variable, une matrice de corrélation a été calculée. Elle permet de s'assurer qu'aucune caractéristique n'est redondante (colinéarité) et identifie les relations éventuelles avec la variable cible (Label).

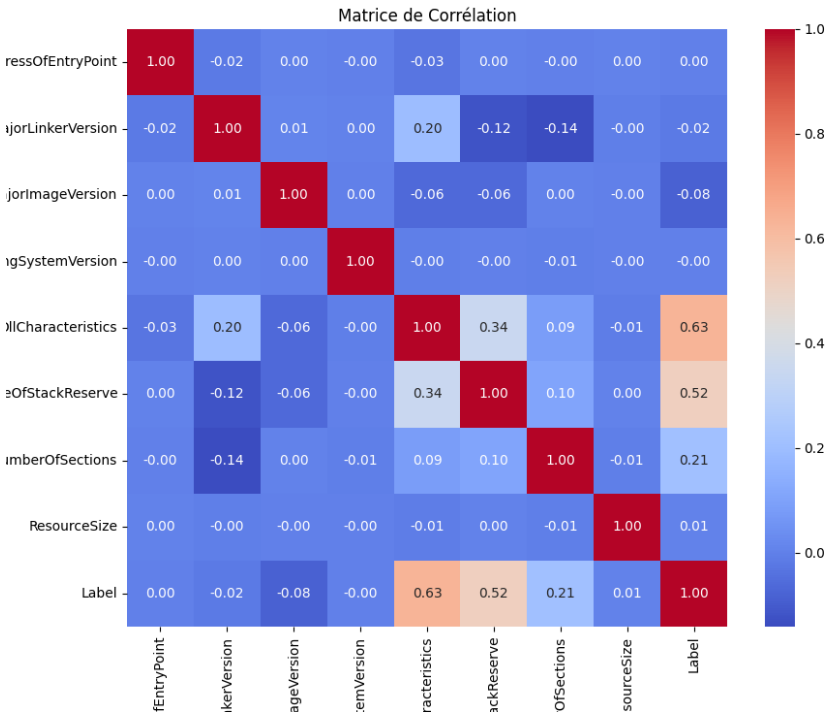

FIGURE 3 – *Matrice de corrélation des caractéristiques extraites*

2.2 Prétraitement et Normalisation

Les données extraites présentent des échelles très disparates (la taille se compte en centaines de milliers d'octets, tandis que l'entropie varie de 0 à 8). Les algorithmes basés sur le calcul de distances (comme SVM et KNN) étant très sensibles à ces différences d'échelle, une étape de **Normalisation** (Standardisation) des données a été réalisée :

$$z = \frac{x - \mu}{\sigma}$$

Cette transformation centre les variables sur une moyenne de 0 avec un écart-type de 1. Nous avons utilisé le `StandardScaler` de *scikit-learn*, dont l'état a été sauvegardé (`scaler.pkl`) pour l'appliquer uniformément lors de l'inférence.

3 Étude Comparative Multimodèle

Afin de déterminer la meilleure architecture algorithmique pour la détection, une phase d'entraînement et d'évaluation comparative a été menée. Les données ont été séparées en ensembles d'entraînement (70%) et de test (30%) avec stratification pour respecter l'équilibre des classes.

3.1 Les Modèles Évalués

Trois architectures algorithmiques ont été mises en concurrence :

1. **Support Vector Machine (SVM)** : Reconnu pour son efficacité dans les espaces de grande dimension et sa capacité à trouver l'hyperplan séparateur optimal par maximisation de la marge.
2. **Random Forest** : Modèle ensembliste robuste, particulièrement adapté au traitement de données tabulaires et résistant à l'overfitting.
3. **K-Nearest Neighbors (KNN)** : Approche intuitive se basant sur la proximité géométrique des caractéristiques dans l'espace vectoriel.

3.2 Résultats de l'Évaluation

La métrique clé retenue pour l'évaluation est le **F1-Score**, car il offre un compromis optimal entre le Rappel (détection exhaustive des menaces) et la Précision (limitation des faux positifs) :

$$F1 = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Les résultats obtenus sur l'ensemble de test (30% des données) montrent d'excellentes performances globales. Le tableau et le graphique comparatifs ci-dessous mettent en évidence ces résultats :

Modèle	F1-Score	Précision	Rappel
SVM	0.9685	0.9621	0.9750
Random Forest	**0.9932**	**0.9938**	**0.9926**
KNN	0.9851	0.9861	0.9840

TABLE 1 – *Comparaison des performances des modèles sur le test set étendu*

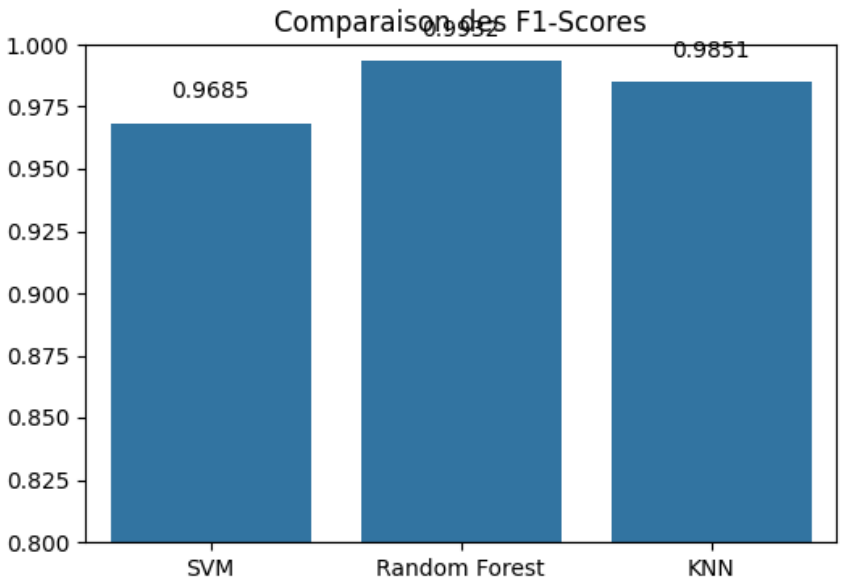

FIGURE 4 – *Comparatif visuel des F1-Scores*

L'évaluation met clairement en évidence la supériorité du **Random Forest**, avec un F1-Score exceptionnel de **0.9932**. Contrairement aux conclusions tirées sur le premier jeu de données synthétique restreint, c'est ce modèle qui capte le mieux la complexité et la non-linéarité des dizaines de descripteurs du nouveau dataset réel. Le **Random Forest** a donc été logiquement désigné *Modèle Champion*.

4 Optimisation du Modèle Champion

Pour atteindre ces performances optimales avec le **Random Forest**, un réglage approfondi de ses hyperparamètres a été mené via la méthode **GridSearchCV**, couplée à une validation croisée à 5 plis (5-fold Cross-Validation).

La grille de recherche a exploré les espaces de paramètres suivants :

— `n_estimators` (nombre d'arbres) : [50, 100, 200]

— `max_depth` (profondeur maximale) : [None, 10, 20, 30]

— `min_samples_split` : [2, 5, 10]

Résultats de l'optimisation : Les paramètres optimaux retenus ont permis de maximiser le F1-Score sur les plis de validation. Le modèle ainsi optimisé est capable de s'adapter finement aux nombreuses features (comme `AddressOfEntryPoint`, `DllCharacteristics`, `SizeOfStackReserve`, etc.) sans sur-apprendre. Ce champion a

été sauvegardé (`malware_classifier.pkl`) avec la liste exacte des features attendues (`features.pkl`) pour garantir la cohérence en inférence.

5 Déploiement et Inférence

Le projet s'achève par le déploiement du modèle dans une architecture orientée micro-services :

1. **Backend FastAPI** : Une API robuste permettant l'upload d'un binaire suspect. Une fonction d'extraction à la volée s'appuyant sur la bibliothèque `pefile` parse l'exécutable pour en extraire ses propriétés structurelles. Celles-ci passent ensuite par le scaler préalablement enregistré avant d'être envoyées au modèle SVM pour prédiction.
2. **Frontend Streamlit** : Une interface utilisateur interactive, avec indicateurs de performances (F1-score, métriques en temps réel) et systèmes d'alertes visuelles (rouge pour malveillant, vert pour sain).

6 Conclusion

L'approche de Machine Learning par analyse statique s'avère extrêmement pertinente pour le triage rapide et sans danger de fichiers exécutables. L'adaptation de l'architecture logicielle au nouveau jeu de données réel a permis de sélectionner un modèle **Random Forest** d'une grande fiabilité. L'alignement rigoureux entre les caractéristiques extraites par l'API via `pefile` et celles ayant servi à l'entraînement, ainsi que le design soigné de l'interface Streamlit, font de ce projet une preuve de concept aboutie pour des systèmes Endpoint Detection and Response (EDR) intelligents de nouvelle génération.